

KUBERNETES-08

1. Create and Test a Kubernetes Pod with an EmptyDir Volume

Step 1: Create YAML

```
apiVersion: v1
kind: Pod
metadata:
  name: emptydir-pod
spec:
  containers:
    - name: writer
      image: nginx:1.27
      command: ["/bin/sh", "-c"]
      args:
        - |
          echo "Hello from emptyDir volume" > /data/message.txt
          sleep 3600
      volumeMounts:
        - name: shared-storage
          mountPath: /data

    - name: reader
      image: busybox
      command: ["/bin/sh", "-c"]
      args:
        - |
          sleep 10
          cat /data/message.txt
          sleep 3600
      volumeMounts:
        - name: shared-storage
          mountPath: /data

  volumes:
    - name: shared-storage
      emptyDir: {}
```

Step 2: Create Pod

Apply & Check

```
ubuntu@k8s-master:~$ vi emptydir-pod.yaml
ubuntu@k8s-master:~$ kubectl apply -f emptydir-pod.yaml
pod/emptydir-pod created
ubuntu@k8s-master:~$ kubectl get pod emptydir-pod
NAME          READY   STATUS    RESTARTS   AGE
emptydir-pod  2/2     Running   0           7s
ubuntu@k8s-master:~$
```

Step 3: Check the writer container

```
ubuntu@k8s-master:~$ kubectl exec emptydir-pod -c writer -- cat /data/message.txt
Hello from emptyDir volume
ubuntu@k8s-master:~$
```

Step 4: Check the reader container

```
ubuntu@k8s-master:~$ kubectl exec emptydir-pod -c reader -- cat /data/message.txt
Hello from emptyDir volume
ubuntu@k8s-master:~$
```

What did we prove?

The writer created:

/data/message.txt

The reader could read the same file.

Step 5: Test persistence during container restart

You can delete the writer container's process indirectly by restarting the Pod, but note that restarting the **Pod** creates a new emptyDir.

The important distinction is:

Container restart → emptyDir remains

Pod deletion → emptyDir deleted

Step 7: Delete Pod

```
ubuntu@k8s-master:~$ kubectl delete pod emptydir-pod
pod "emptydir-pod" deleted
ubuntu@k8s-master:~$
```

Now create it again:

```
ubuntu@k8s-master:~$ kubectl apply -f emptydir-pod.yaml
pod/emptydir-pod created
ubuntu@k8s-master:~$
```

The old data is gone because a new emptyDir was created.

Interview answer

emptyDir provides temporary storage for a Pod. It can be shared between containers in the same Pod, and its data is deleted when the Pod is removed.

2. Configure a HostPath Volume in Kubernetes and Validate Data Persistence

Step 1: Create directory on a worker node

```
ubuntu@k8s-master:~$ kubectl get nodes
NAME           STATUS    ROLES    AGE   VERSION
k8s-master     Ready     control-plane  9d   v1.29.15
worker01       Ready     <none>      9d   v1.29.15
worker02       Ready     <none>      9d   v1.29.15
```

Create the directory on the worker node where the Pod will run.

On worker1

```
ubuntu@worker01:~$ sudo mkdir -p /data/k8s
sudo chmod 777 /data/k8s
ubuntu@worker01:~$
```

Step 2: Create YAML

On master:

```

apiVersion: v1
kind: Pod
metadata:
  name: hostpath-pod
spec:
  nodeName: Worker-01

  containers:
  - name: nginx
    image: nginx:1.27
    command: ["/bin/sh", "-c"]
    args:
    - |
      echo "Data stored on hostPath" > /data/message.txt
      sleep 3600

    volumeMounts:
    - name: host-storage
      mountPath: /data

  volumes:
  - name: host-storage
    hostPath:
      path: /data/k8s
      type: Directory

```

Step 3: Create Pod & Check:

```

ubuntu@k8s-master:~$ vi hostpath-pod.yaml
ubuntu@k8s-master:~$ kubectl apply -f hostpath-pod.yaml
pod/hostpath-pod created
ubuntu@k8s-master:~$ kubectl get pod hostpath-pod -o wide

```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
hostpath-pod	1/1	Running	0	9s	10.244.1.87	worker01	<none>		<none>	

Step 4: Check data inside Pod

```

ubuntu@k8s-master:~$ kubectl exec hostpath-pod -- cat /data/message.txt
Data stored on hostPath
ubuntu@k8s-master:~$

```

Step 5: Check data directly on worker

into worker01

```

ubuntu@worker01:~$ sudo cat /data/k8s/message.txt
Data stored on hostPath
ubuntu@worker01:~$

```

Step 6: Delete Pod

```

ubuntu@k8s-master:~$ kubectl delete pod hostpath-pod
pod "hostpath-pod" deleted
ubuntu@k8s-master:~$

```

Step 7: Check node data again

to worker01:

```

ubuntu@worker01:~$ sudo cat /data/k8s/message.txt
Data stored on hostPath
ubuntu@worker01:~$

```

The data should still exist.

Why?

Because the Pod was deleted, but:

/data/k8s

is a directory on the worker node.

Important limitation of hostPath

Suppose the Pod was using:

worker01

and worker01 goes down.

Kubernetes may schedule the Pod on:

worker02

But worker02 may not have:

/data/k8s

with the same data.

Therefore hostPath is generally **not suitable for portable production persistent storage**.

Interview answer

hostPath mounts a directory from the Kubernetes node into a Pod. The data can survive Pod deletion, but it is tied to that particular node.

3. Deploy an Amazon EBS Volume Using Persistent Volume and Persistent Volume Claim (PVC)

What is PV?

PV = **PersistentVolume**

It represents storage available to Kubernetes.

Example:

PV

capacity: 10Gi

storageClass: ebs

What is PVC?

PVC = **PersistentVolumeClaim**

It is a request for storage.

For example:

"I need 5Gi storage."

Kubernetes finds a suitable PV.

Important AWS point

EBS is **block storage** and is normally attached to one Availability Zone.

Your current cluster uses EC2 nodes, so make sure the EBS volume is in the **same Availability Zone as the node** that will use it.

For a modern Kubernetes cluster, the preferred approach is usually the **AWS EBS CSI driver + StorageClass + dynamically provisioned PVC**, rather than manually creating an old-style awsElasticBlockStore PV.

For your practice lab, we'll demonstrate the PV/PVC concept using the EBS CSI driver.

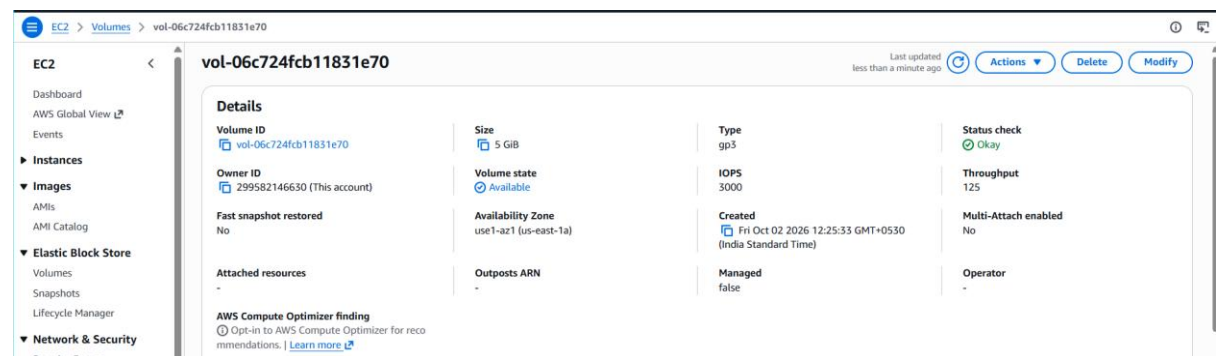
Step 1: Create an EBS volume in AWS

Go to:

AWS Console → EC2 → Elastic Block Store → Volumes → Create volume

Very important

Your EBS volume must be in the **same Availability Zone** as the worker node where your Pod will run



Step 2: Check your Kubernetes nodes

```
ubuntu@k8s-master:~$ kubectl get nodes -o wide
NAME          STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
k8s-master    Ready     control-plane   9d    v1.29.15   172.31.4.10   <none>         Ubuntu 26.04 LTS     7.0.0-1013-aws   containerd://2.3.5
worker01      Ready     <none>        9d    v1.29.15   172.31.4.11   <none>         Ubuntu 26.04 LTS     7.0.0-1006-aws   containerd://2.3.5
worker02      Ready     <none>        9d    v1.29.15   172.31.4.12   <none>         Ubuntu 26.04 LTS     7.0.0-1006-aws   containerd://2.3.5
```

Step 3: Create the PersistentVolume

Create the YAML file:

```
GNU nano 8.7.1 pv.yaml *
apiVersion: v1
kind: PersistentVolume
metadata:
  name: ebs-pv
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  awsElasticBlockStore:
    volumeID: vol-06c724fcb11831e70
    fsType: ext4
```

Step 4: Create the PV & Check:

```
ubuntu@k8s-master:~$ nano pv.yaml
ubuntu@k8s-master:~$ kubectl apply -f pv.yaml
persistentvolume/ebs-pv created
ubuntu@k8s-master:~$ kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS	VOLUMEATTRIBUTESCLASS	REASON	AGE
ebs-pv	5Gi	RWO	Retain	Available			<unset>		13s

```
ubuntu@k8s-master:~$
```

Step 5: Create the PVC

```
GNU nano 8.7.1 pvc.yaml *
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: ebs-pvc
spec:
  accessModes:
    - ReadWriteOnce

  resources:
    requests:
      storage: 5Gi
```

Create it:

```
ubuntu@k8s-master:~$ nano pvc.yaml
ubuntu@k8s-master:~$ kubectl apply -f pvc.yaml
persistentvolumeclaim/ebs-pvc created
ubuntu@k8s-master:~$
```

Step 6: Check the PVC

```
ubuntu@k8s-master:~$ kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	VOLUMEATTRIBUTESCLASS	AGE
ebs-pvc	Bound	ebs-pv	5Gi	RWO		<unset>	82s

```
ubuntu@k8s-master:~$
```

The important part is:

STATUS = Bound

This means:

PVC successfully connected to PV

Also check:

kubectl get pv

```
ubuntu@k8s-master:~$ kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS	VOLUMEATTRIBUTESCLASS	REASON	AGE
ebs-pv	5Gi	RWO	Retain	Bound	default/ebs-pvc		<unset>		5m17s

```
ubuntu@k8s-master:~$
```

EBS Volume

↓

PV

↓

PVC

is connected.

Step 7: Create a Pod using the PVC

```
GNU nano 8.7.1 ebs-pod.yaml *
apiVersion: v1
kind: Pod
metadata:
  name: ebs-pod
spec:
  nodeName: worker01

  containers:
  - name: nginx
    image: nginx

    volumeMounts:
    - name: ebs-storage
      mountPath: /usr/share/nginx/html

  volumes:
  - name: ebs-storage
    persistentVolumeClaim:
      claimName: ebs-pvc
```

Important line

nodeName: worker01

This tells Kubernetes:

Run this Pod on worker01.

We are doing this because your EBS volume is created in a particular AWS Availability Zone.

Step 8: Create the Pod

Run:

kubectl apply -f ebs-pod.yaml

check the file:

```
[ec2-user@k8s-master ~]$ kubectl get pod ebs-app-pod -w
NAME          READY   STATUS             RESTARTS   AGE
ebs-app-pod   0/1     ContainerCreating   0           12s
ebs-app-pod   1/1     Running             0           13s
^C[ec2-user@k8s-master ~]kubectl exec ebs-app-pod -- cat /data/test.txt
Hello from AWS EBS Storage
[ec2-user@k8s-master ~]$
```

4. Set Up an Amazon EFS Volume and Attach it to Multiple Pods

Step 1: Create Security Group for EFS

Create an EFS Security Group and allow NFS TCP port 2049 from the Kubernetes VPC/cluster CIDR.

Rule:

Type: NFS

Protocol: TCP

Port: 2049

Source: Kubernetes VPC CIDR

EC2 > Security Groups > sg-0ef8f7d13086c12ac > Edit inbound rules

Edit inbound rules Info

Inbound rules control the incoming traffic that's allowed to reach the instance.

Inbound rules Info	Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info
Security group rule ID					
sgr-0019b05190539ee4b	NFS	TCP	2049	Custom	

172.31.0.0/16 X

Delete

Step 2: Create Amazon EFS File System

Create an Amazon EFS File System in the same VPC as the Kubernetes worker nodes.

Create mount targets for the Availability Zones where the Kubernetes nodes are running.

Amazon EFS > File systems

Elastic File System

File systems
Access points

AWS Backup [↗](#)
AWS DataSync [↗](#)
AWS Transfer [↗](#)
Documentation [↗](#)

Success! File system (fs-03df9f02a21c7c5b9) is available. [View file system](#) X

File systems (1) [View details](#) [Delete](#) [Create file system](#)

Filter by property values

	Name	File system ID	Encrypt in transit	Total size	Size in Standard	Size in IA	Size in Archive	Provisioned Throughput (MB/s)	File system state
	k8s-shared-efs	fs-03df9f02a21c7c5b9	Encrypt in transit	6.00 KIB	6.00 KIB	0 Bytes	0 Bytes	-	Available

Step 3: Verify EFS CSI Driver

Verify that the EFS CSI driver is installed in the cluster.

```
[ec2-user@k8s-master ~]$ kubectl apply -k "github.com/kubernetes-sigs/aws-efs-csi-driver/deploy/kubernetes/overlays/stable/?ref=release-1.7"
# Warning: 'bases' is deprecated. Please use 'resources' instead. Run 'kustomize edit fix' to update your Kustomization automatically.
serviceaccount/efs-csi-controller-sa created
serviceaccount/efs-csi-node-sa created
clusterrole.rbac.authorization.k8s.io/efs-csi-external-provisioner-role created
clusterrole.rbac.authorization.k8s.io/efs-csi-external-provisioner-role-describe-secrets created
clusterrole.rbac.authorization.k8s.io/efs-csi-node-role created
rolebinding.rbac.authorization.k8s.io/efs-csi-provisioner-binding-describe-secrets created
clusterrolebinding.rbac.authorization.k8s.io/efs-csi-node-binding created
clusterrolebinding.rbac.authorization.k8s.io/efs-csi-provisioner-binding created
deployment.apps/efs-csi-controller created
daemonset.apps/efs-csi-node created
csidriver.storage.k8s.io/efs.csi.aws.com created
[ec2-user@k8s-master ~]$
```

Step 4: Create StorageClass

Create efs-storageclass.yaml:

apiVersion: storage.k8s.io/v1

kind: StorageClass

metadata:

name: efs-sc

provisioner: efs.csi.aws.com

Apply:

```
kubectl apply -f efs-storageclass.yaml
```

Verify:

```
kubectl get storageclass
```

Step 5: Create PersistentVolume

Create efs-pv.yaml:

```
apiVersion: v1
```

```
kind: PersistentVolume
```

```
metadata:
```

```
  name: efs-pv
```

```
spec:
```

```
  capacity:
```

```
    storage: 5Gi
```

```
  volumeMode: Filesystem
```

```
  accessModes:
```

```
    - ReadWriteMany
```

```
  persistentVolumeReclaimPolicy: Retain
```

```
  storageClassName: efs-sc
```

```
  csi:
```

```
    driver: efs.csi.aws.com
```

```
    volumeHandle: <EFS-FILE-SYSTEM-ID>
```

Replace:

```
<EFS-FILE-SYSTEM-ID>
```

with your EFS File System ID.

Apply:

```
kubectl apply -f efs-pv.yaml
```

Step 6: Create PersistentVolumeClaim

Create efs-pvc.yaml:

```
apiVersion: v1
```

```
kind: PersistentVolumeClaim
```

metadata:

name: efs-pvc

spec:

accessModes:

- ReadWriteMany

storageClassName: efs-sc

resources:

requests:

storage: 5Gi

Apply:

kubectl apply -f efs-pvc.yaml

Verify:

kubectl get pv

kubectl get pvc

Expected:

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES
------	--------	--------	----------	--------------

efs-pvc	Bound	efs-pv	5Gi	RWX
---------	-------	--------	-----	-----

```
ec2-user@k8s-master:~  
apiVersion: storage.k8s.io/v1  
kind: StorageClass  
metadata:  
  name: efs-sc  
provisioner: efs.csi.aws.com  
---  
apiVersion: v1  
kind: PersistentVolume  
metadata:  
  name: efs-pv  
spec:  
  capacity:  
    storage: 5Gi  
  volumeMode: Filesystem  
  accessModes:  
    - ReadWriteMany # Allows multiple pods across nodes to write simultaneously  
  persistentVolumeReclaimPolicy: Retain  
  storageClassName: efs-sc  
  csi:  
    driver: efs.csi.aws.com  
    volumeHandle: fs-03df9f02a21c7c5b9  
---  
apiVersion: v1  
kind: PersistentVolumeClaim  
metadata:  
  name: efs-pvc  
spec:  
  accessModes:  
    - ReadWriteMany  
  storageClassName: efs-sc  
  resources:  
    requests:  
      storage: 5Gi
```

```
[ec2-user@k8s-master ~]$ kubectl apply -f efs-storage.yaml
storageclass.storage.k8s.io/efs-sc created
persistentvolume/efs-pv created
persistentvolumeclaim/efs-pvc created
[ec2-user@k8s-master ~]$ kubectl get sc,pv,pvc
NAME                                PROVISIONER      RECLAIMPOLICY   VOLUMEINTEGRITYMODE   ALLOWVOLUMEEXPANSION   AGE
storageclass.storage.k8s.io/efs-sc  ebs.csi.aws.com  Delete          WaitForFirstConsumer   false                  20h
storageclass.storage.k8s.io/efs-sc  ebs.csi.aws.com  Delete          Immediate              false                  13s
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS	VOLUMEATTRIBUTESCLASS	REASON	AGE
persistentvolume/efs-pv	5Gi	RWX	Retain	Bound	default/efs-pvc	efs-sc	<unset>		13s
persistentvolume/pvc-0c49781a-e754-4810-97b4-458335d934a7	4Gi	RWO	Delete	Bound	default/ebs-mysql-pvc	ebs-sc	<unset>		3h56m

```
NAME                                STATUS    VOLUME                                CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
persistentvolumeclaim/ebs-mysql-pvc  Bound     pvc-0c49781a-e754-4810-97b4-458335d934a7  4Gi        RWO            ebs-sc         <unset>                 20h
persistentvolumeclaim/efs-pvc        Bound     efs-pv                                     5Gi        RWX            efs-sc         <unset>                 13s
```

Step 7: Create Two Pods Using the Same PVC

Attach the EFS PVC to Multiple Pods (ReadWriteMany)

EFS allows concurrent access. Create two distinct Pods sharing the exact same PVC

```
[ec2-user@k8s-master ~]$ kubectl apply -f multi-pod-efs.yaml
pod/efs-writer-pod-1 created
pod/efs-writer-pod-2 created
[ec2-user@k8s-master ~]$
```

Read the shared file from efs-writer-pod-1 and read the same shared file from efs-writer-pod-2

```
[ec2-user@k8s-master ~]$ kubectl exec efs-writer-pod-2 -- cat /mnt/efs/shared.txt
E1002 11:20:36.036170 33887 websocket.go:296] Unknown stream id 1, discarding message
Written by Pod 2 at Fri Oct 2 11:19:07 UTC 2026
Written by Pod 1 at Fri Oct 2 11:19:07 UTC 2026
Written by Pod 2 at Fri Oct 2 11:19:12 UTC 2026
Written by Pod 1 at Fri Oct 2 11:19:12 UTC 2026
Written by Pod 2 at Fri Oct 2 11:19:17 UTC 2026
Written by Pod 1 at Fri Oct 2 11:19:17 UTC 2026
Written by Pod 2 at Fri Oct 2 11:19:22 UTC 2026
Written by Pod 1 at Fri Oct 2 11:19:22 UTC 2026
Written by Pod 2 at Fri Oct 2 11:19:27 UTC 2026
Written by Pod 1 at Fri Oct 2 11:19:27 UTC 2026
```

```
[ec2-user@k8s-master ~]$ kubectl exec efs-writer-pod-1 -- cat /mnt/efs/shared.txt
Written by Pod 2 at Fri Oct 2 11:19:07 UTC 2026
Written by Pod 1 at Fri Oct 2 11:19:07 UTC 2026
Written by Pod 2 at Fri Oct 2 11:19:12 UTC 2026
Written by Pod 1 at Fri Oct 2 11:19:12 UTC 2026
Written by Pod 2 at Fri Oct 2 11:19:17 UTC 2026
Written by Pod 1 at Fri Oct 2 11:19:17 UTC 2026
Written by Pod 2 at Fri Oct 2 11:19:22 UTC 2026
Written by Pod 1 at Fri Oct 2 11:19:22 UTC 2026
Written by Pod 2 at Fri Oct 2 11:19:27 UTC 2026
Written by Pod 1 at Fri Oct 2 11:19:27 UTC 2026
Written by Pod 2 at Fri Oct 2 11:19:32 UTC 2026
Written by Pod 1 at Fri Oct 2 11:19:32 UTC 2026
[ec2-user@k8s-master ~]$
```

Here, both Pods continuously read and append entries to the same /mnt/efs/shared.txt file in real-time, demonstrating shared ReadWriteMany persistence across pods

5. Implement Liveness and Readiness Probes

Step 1: Create Pod Manifest

Create probed-pod.yaml:

```
ec2-user@k8s-master:~  
apiVersion: v1  
kind: Pod  
metadata:  
  name: probe-test-pod  
  labels:  
    app: probe-test  
spec:  
  containers:  
    - name: nginx-probe  
      image: nginx:alpine  
      # Create /tmp/healthy, wait 30 seconds, delete it, then sleep to keep process alive  
      command: ["/bin/sh", "-c", "touch /tmp/healthy; sleep 30; rm -f /tmp/healthy; sleep 600"]  
      ports:  
        - containerPort: 80  
      readinessProbe:  
        httpGet:  
          path: /  
          port: 80  
          initialDelaySeconds: 5  
          periodSeconds: 5  
      livenessProbe:  
        exec:  
          command:  
            - cat  
            - /tmp/healthy  
          initialDelaySeconds: 5  
          periodSeconds: 5
```

The readiness probe checks /ready. Since the default Nginx image does not provide this page, the readiness probe will eventually fail. The liveness probe checks whether /tmp/healthy exists.

Step 2: Create the Health File

Apply the Pod:

```
kubectl apply -f probed-pod.yaml
```

Check the Pod:

```
kubectl get pod probe-test-pod -w
```

Initially, the Pod may show:

READY STATUS

0/1 Running

```
[ec2-user@k8s-master ~]$ kubectl apply -f probe-pod.yaml  
kubectl get pod probe-test-pod -w  
pod/probe-test-pod created  
NAME          READY   STATUS             RESTARTS   AGE  
probe-test-pod 0/1     Terminating      0          0s  
probe-test-pod 0/1     ContainerStatusUnknown 0          2s  
probe-test-pod 0/1     ContainerStatusUnknown 0          2s  
probe-test-pod 0/1     ContainerStatusUnknown 0          2s  
^C[ec2-user@k8s-master ~]$ |
```

Step 3: Check Probe Status

View the Pod details:

```
kubectl describe pod probed-pod
```

Check the Events section for readiness probe results.

The readiness probe failure causes the Pod to remain:

0/1

But the container continues running.

```
Events:
  Type     Reason      Age          From          Message
  ----     -
Normal    Scheduled   34m         default-scheduler   Successfully assigned default/probe-test-pod to ip-172-31-30-184.ec2.internal
Warning   Unhealthy   33m (x3 over 33m)   kubelet           Liveness probe failed: cat: can't open '/tmp/healthy': No such file or directory
Normal    Killing     33m         kubelet           Container nginx-probe failed liveness probe, will be restarted
Normal    Pulled      33m (x2 over 34m)   kubelet           Container image "nginx:alpine" already present on machine
Normal    Created     33m (x2 over 34m)   kubelet           Created container: nginx-probe
Normal    Started     29m (x5 over 34m)   kubelet           Started container nginx-probe
```

Step 4: Verify Liveness

Check the restart count:

```
kubectl get pod probed-pod
```

Expected:

READY STATUS RESTARTS

0/1 Running 0

The readiness probe failure does not restart the container.

The liveness probe continues checking:

/tmp/healthy

Because the file exists only if created, the liveness status can be tested by creating/removing the file inside the container.

For example:

```
kubectl exec probed-pod -- touch /tmp/healthy
```

Then verify:

```
kubectl get pod probed-pod
```

```
^C[ec2-user@k8s-master ~]kubectl exec probe-test-pod -c nginx-probe -- rm /usr/share/nginx/html/index.htmlm
[ec2-user@k8s-master ~]$ kubectl get pod probe-test-pod
NAME          READY   STATUS    RESTARTS   AGE
probe-test-pod 0/1     Running   0           41s
[ec2-user@k8s-master ~]$
```